City of the World Dashboard Technical Report

DATA ANALYTICS FOR FINANCE AND INSURANCE
Prof. Leonardo Felician
MBA 36
MIB Trieste

Team Four-mula (4): Daniele Recanati, Tito Rodda, Germán Oviedo, Ensar Omerhodzic
May 2025

Table of Contents

Executive Summary

The Cities of the World Dashboard is a fully automated data pipeline that transforms a single user input, a city name, into a comprehensive, multi-dimensional intelligence record. Designed on the principle of **operational leverage**, the system eliminates manual data gathering and replaces it with an orchestrated enrichment engine capable of delivering geographic, meteorological, cultural, visual, and corporate intelligence in seconds.

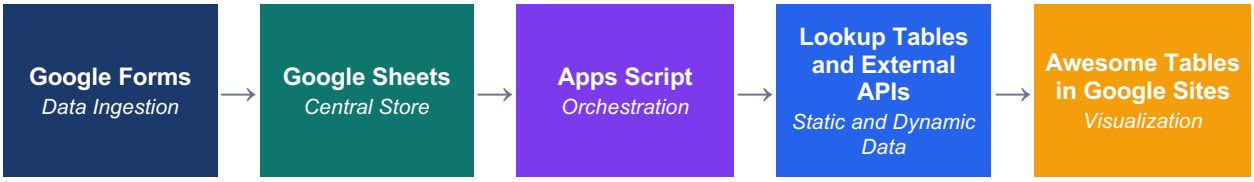
By combining Google Forms, Google Sheets, Google Apps Script, DuckDB, Wikipedia/Wikidata APIs, a live Weather API, and Google Gemini, the pipeline achieves a rare balance: it is simultaneously low-cost, low-infrastructure, and high-output. The architectural philosophy is captured in three principles:

- **Store what is stable.**
- **Calculate what changes.**
- **Automate the connection between both.**

Key Performance Indicators

7	23+	3	0
Pipeline Stages	Data Fields per City	External APIs	Manual Steps Required

Technology Stack



The pipeline runs entirely within the Google ecosystem (Awesome Tables is kind of an exception, but it is meant to be used with Sheets so that point is moot), eliminating the need for external servers, custom backends, or complex deployment workflows. External enrichments are handled through lightweight API integrations triggered by an Apps Script event handler.

Step 1 - Data Ingestion & Centralisation

The user-facing entry point is a standardised **Google Form** that captures a single field: the city name. Upon submission, the response is automatically routed to the ***Form Responses 1*** worksheet of a linked Google Sheet, creating a new row for each submission. This triggers the entire downstream pipeline without any manual intervention.

The decision to remain within the Google ecosystem was deliberate. Google Forms, Sheets, and Apps Script are tightly integrated, allowing data ingestion, transformation, enrichment, and visualisation to operate in a single familiar environment — without external databases, ETL tools, or infrastructure management.

Ingestion Architecture

Component	Role	Output
Google Forms	Standardised user input interface	*Form submission event*
Form Responses 1	Primary operational worksheet	*New row per city*
onFormSubmit(e)	Apps Script event trigger	*Pipeline execution start*

Step 2 - Database Engineering & Lookup Tables

Rather than querying external APIs for every field on every submission, the pipeline leverages a pre-computed relational database sourced from an open-source repository (https://github.com/dr5hn/countries-states-cities-database). This database contains city, state, and country records including population figures, coordinates, ISO codes, capital cities, and Wikidata IDs.

The raw database consists of three separate CSV files (cities, states, countries). To create a single unified lookup table, DuckDB was used to execute an in-process SQL join. The resulting dataset was exported as a CSV and imported into Google Sheets as a dedicated *city_data* worksheet, effectively a local relational database.

DuckDB Consolidation Query

```
SELECT
  c.id          AS city_id,
  c.name        AS city_name,
  s.name        AS state_name,
  co.name       AS country_name,
  co.region     AS continent,
  co.subregion  AS subregion,
  co.iso2       AS country_iso2,
  co.iso3       AS country_iso3,
  c.latitude,
  c.longitude,
  c.population  AS city_population,
  co.population AS country_population,
  co.capital    AS country_capital,
  c.wikiDataId  AS city_wikidata_id,
  co.wikiDataId AS country_wikidata_id
FROM cities c
LEFT JOIN states s  ON c.state_id  = s.id
LEFT JOIN countries co ON c.country_id = co.id;
```

The joined result provides 15 fields per city in a single flat lookup structure. Because city and country metadata changes infrequently, storing it locally eliminates API latency for every subsequent query, significantly improving response time and resilience.

Allianz Offices data

For the Allianz data, a similar precomputed scraping approach was used (https://github.com/mrbungie/allianz-scraper) over their contact information page (https://www.allianz.com/en/about-us/company/contact.html), then those results were uploaded as an extra sheet and accessed via XLOOKUP to add data about Allianz offices to each new added city.

Field Mapping

The Apps Script trigger writes XLOOKUP formulas into each new row, mapping the submitted city name to the pre-computed values in the city_data worksheet. The complete column mapping is shown below:

Column	Field	Source	Method
B	City (user input)	Google Forms	*Form submission*
C	City ID	city_data worksheet	*XLOOKUP (inyected via Script)*
D	State	city_data worksheet	*XLOOKUP (inyected via Script)*
E	Country	city_data worksheet	*XLOOKUP (inyected via Script)*
F	Continent	city_data worksheet	*XLOOKUP (inyected via Script)*
G	Subregion	city_data worksheet	*XLOOKUP (inyected via Script)*
H	Country ISO2	city_data worksheet	*XLOOKUP (inyected via Script)*
I	Country ISO3	city_data worksheet	*XLOOKUP (inyected via Script)*
J	Latitude	city_data worksheet	*XLOOKUP (inyected via Script)*
K	Longitude	city_data worksheet	*XLOOKUP (inyected via Script)*
L	City Population	city_data worksheet	*XLOOKUP (inyected via Script)*
M	Country Population	city_data worksheet	*XLOOKUP (inyected via Script)*
N	Country Capital	city_data worksheet	*XLOOKUP (inyected via Script)*
O	City Wikidata ID	city_data worksheet	*XLOOKUP (inyected via Script)*
P	Country Wikidata ID	city_data worksheet	*XLOOKUP (inyected via Script)*
Q	Flag	city_data worksheet	*XLOOKUP (inyected via Script)*
R	Airport	Google Gemini	*Apps Script (AI live query)*
S	Greeting	Google Gemini	*Apps Script (AI live query)*
T	Motto	Wikidata API	*Apps Script (wikidata live query)*
U	Coat of Arms	Wikidata API	*Apps Script (wikidata live query)*
V–Y	Weather (condition, label, symbol, timestamp)	Weather API	*Apps Script (live query)*
Z	City Photo	Wikipedia API	*Apps Script (wikidata live query)*
AA	Tooltip (Awesome Tables)	Awesome Tables template	*Empty (AT placeholder)*
AB	Allianz Company	allianz_offices worksheet	*XLOOKUP (inyected via Script)*
AC	Allianz Address	allianz_offices worksheet	*XLOOKUP (inyected via Script)*
AD	Allianz Telephone	allianz_offices worksheet	*XLOOKUP (inyected via Script)*

Step 3 - Data Strategy: Static vs. Dynamic

A central architectural decision was the division of all data fields into two categories, each handled by a distinct strategy. This separation is what enables the pipeline to be both fast and current — delivering stable reference data instantly while always retrieving time-sensitive information live.

STATIC / PRE-CALCULATED DATA	DYNAMIC / LIVE DATA
Storage Strategy	**Storage Strategy**
Pre-calculated once, stored in Google Sheets. Retrieved instantly via XLOOKUP.	Calculated on each form submission using live API calls.
Data Fields	**Data Fields**
City ID, State, Country	Current Weather (status, label, icon), using https://api.met.no/weatherapi/locationforecast/
Continent & Subregion	AI-generated Airport Name using Gemini API
ISO2 / ISO3 Country Codes	AI-generated Local Greeting using Gemini API
Latitude & Longitude	
City & Country Population	
Country Capital & Flag	
City & Country Wikidata ID	
City Motto & Coat of Arms	
City Photo (Wikipedia), calculated with sheet formula	
Allianz Office Data	
Source: DuckDB-joined CSV lookup table	*Source: Weather API + Google Gemini*

Amortising processing with storage is the guiding principle. Static data is calculated once using DuckDB and stored permanently. Dynamic data is calculated live on each submission. This minimises both API dependency and computational cost while ensuring the dashboard is always accurate.

Step 4 - API & Script Enrichments

With static geography embedded through lookup tables, the Apps Script trigger orchestrates a series of sequential API calls to enrich each new city record with live and semi-structured context. Each enrichment step writes its result directly to the sheet as soon as it is available, enabling incremental dashboard population without waiting for the entire pipeline to complete.

All can be seen on: https://gist.github.com/mrbungie/d35eabdabc6abd4665e30657dac60f10

Pipeline Execution Sequence

▼ STEP 1 **Static Lookup Formulas**
XLOOKUP writes city/country/Allianz data from pre-computed worksheets

▼ STEP 2 **City Photo Retrieval**
Wikipedia Summary API returns the primary city image; fallback URL used if not found

▼ STEP 3 **Live Weather Enrichment**
Weather API called with lat/long coordinates: status, label, icon, and timestamp written

▼ STEP 4 **Motto & Coat of Arms**
Wikidata queried using the city QID for motto text and coat of arms image URL

▼ STEP 5 **AI-Generated Fields**
Gemini API generates structured JSON containing airport name and localised greeting

STEP 6 **Row Complete**
Dashboard row fully populated; execution logged for monitoring

Enrichment Detail

Step	Function / API	Fields Written	Notes
Photo via Wikidata	getWikipediaCityImage()	City photo URL	*Falls back to placeholder URL*
Weather via api.met.no	GETWEATHERSTATUS / ICON / LABEL	Status, label, icon, timestamp	*Live call, get latest weather report for the new submission if possible (sometimes the API is overloaded)*
Motto via Wikidata	GET_WIKI_PROPERTY(qid, 'motto', 'text')	City motto	*Empty value if missing*
Coat of Arms via Wikidata	GET_WIKI_PROPERTY(qid, 'coat_of_arms', 'image')	Coat of arms image URL	*Empty value if missing*
Airport via Gemini	getCityAirportAndGreeting_()	Main airport name	*Gemini returns structured JSON*
Greeting via Gemini	getCityAirportAndGreeting_()	Localised greeting	*Gemini returns structured JSON, language matched to city/country*

Step 5 - Generative AI Integration

The pipeline incorporates **Google Gemini** to generate information that is either absent from structured databases or impractical to maintain as static data. Specifically, Gemini is used to identify the primary internationally recognised airport for each city and to produce a culturally appropriate localised greeting.

An important implementation lesson was discovered during development. The initial approach used Gemini formulas directly inside Google Sheets cells since we had a Gemini Pro account. However, these formulas did not reliably recalculate when triggered by a new form submission, creating a dependency on manual refresh and breaking the automation model.

The solution was to migrate all Gemini calls into the Apps Script workflow, where the API is called directly and the result is written programmatically. Gemini is instructed to return **structured JSON** containing both the airport and greeting fields, which the script then parses and writes to the appropriate columns. These Gemini calls are not free though, but for such an use case it should not be very expensive.

Gemini JSON Response Schema

```
{
  "airport": "Milan Malpensa Airport (MXP)",
  "greeting": "Benvenuto a Milano!"
}
```

The Gemini step is intentionally placed last in the pipeline. Because it involves a network call to an LLM, it is the slowest enrichment step. Positioning it at the end ensures that faster fields (such as photo, weather, motto, coat of arms) are written to the dashboard first, keeping the partial record useful even while the AI generation completes.

Gemini API Key setup

For managing the API Key secret we use a setGeminiApiKey function that runs once and sets the key using PropertiesService.getScriptProperties().setProperty("GEMINI_API_KEY", apiKey), and then the key string svalue is deleted from the script. Posterior calls use the same PropertiesService to get the GEMINI_API_KEY property.

Step 6 - Business Logic & Error Handling

An automated pipeline operating on free-text user input must anticipate imperfect data. The system incorporates explicit business rules and transparent failure handling to maintain dashboard quality and debuggability regardless of input quality.

Ambiguous City Name Resolution

Many city names are shared across countries and regions. When the XLOOKUP formula matches multiple records, the system applies a deterministic rule: it selects the city with the **highest recorded population**. This is a practical assumption for a prototype environment, as the largest city of a given name is the most statistically likely intended target.

Error Handling Matrix

Failure Scenario	System Response	Value
City name not found in lookup table	Defaults to highest-population city match	*Populated with best match*
Wikipedia image not found	Fallback URL written to cell	*Placeholder image displayed*
Weather API failure	Error caught, fallback written	*Empty string*
Wikidata motto unavailable	Field populated with message	*Empty string*
Coat of arms not in Wikidata	Fallback message written	*Empty string*
Gemini airport/greeting failure	Error handled gracefully	*Empty string*
Gemini returns empty value	Explicit not-found message	*Empty string*

All pipeline steps are wrapped in ***try/catch*** blocks. Rather than failing silently — which would leave cells blank and make errors invisible, the system writes explicit human-readable failure messages. This design philosophy prioritises **visible failure over silent degradation**, making the dashboard easier to audit and debug.

Every execution is logged through ***console.log()*** and ***console.error()*** calls, accessible through the Apps Script execution log. This allows the team to monitor performance, identify slow steps, and trace root causes when individual enrichments fail.

Step 7 - Visualisation: Awesome Tables & Google Sites

With the data pipeline fully operational, the enriched Google Sheet is exposed as a live, interactive map and table through Awesome Tables, a no-code Google Sheets add-on that renders data from a spreadsheet into embeddable, filterable views. The resulting dashboard is embedded in a Google Site.

Awesome Tables Configuration

Awesome Tables reads directly from the *Form Responses 1* worksheet of the linked Google Sheet from range A2:AB. The view is configured as a Maps view, which places each city as a marker on a Google Map using the *Latitude* and *Longitude* columns (J and K). Each marker is rendered as the city's national flag emoji drawn from the *Flag* column (Q). Beneath the map, a searchable and filterable data table lists every submitted city with its Country, Weather, and Company columns visible. Filter controls allow users to narrow results by State, Country, Continent, and Subregion.

This also needs a row that configures each field for Awesome Tables. In this case that corresponds to row 3, which includes markers such as "Hidden" (anything we don't want, "CategoryFilter", "MapsMarker" (the flag), "MapsLat", etc.

Tooltip Template

Clicking a map marker opens a rich pop-up tooltip. Rather than displaying plain text, the tooltip renders a fully styled HTML card built from an Awesome Tables template html and CSS defined in the templates sheet. The HTML template is stored as column A with curley braces definitions (e.g. `{{Field Name}}`) that Awesome Tables substitutes at render time with the corresponding column values for each row. The CSS is defined in column B and used for styling and controlling conditional hidden sections when values are missing. The card structure is as follows:

Card Section	Fields Used
Header Banner	Dark blue (#123a63) background with city `{{Greeting}}` as the main title and `{{Country}} — {{Continent}}` as the subtitle.
Key Statistics Row	Side-by-side display of `{{Population}}` (city population) and `{{Airport}}` (main international airport, Gemini-generated).
Motto Block	Light grey box with a blue left-border accent, displaying `{{Motto}}` in italics. Falls back to "Motto not found in Wikidata" when unavailable.
Weather Summary	Centred weather label `{{Weather}}` with meteorological icon `{{Weather Symbol}}` and a smaller grey timestamp showing `{{Weather Last Update}}`.
Image Strip	Side-by-side images: the city's `{{Coat of Arms}}` (SVG from Wikidata, max 70×95 px) and a city landscape `{{Photo}}` (130×90 px, rounded corners, sourced from Wikipedia).
Company info	Allianz company information like name, address and telephone.

Because Awesome Tables resolves the placeholder variables at display time from the live sheet data, the card always reflects the most recent pipeline run. Any city added via the Google Form appears automatically in the dashboard within seconds of the pipeline completing (after refreshing), with no manual intervention required on the visualisation layer.

Google Sites Embedding

The Awesome Tables view is embedded as an **iframe** inside a Google Site, making the dashboard publicly shareable without requiring any viewer to have a Google account or access to the underlying spreadsheet. The site is hosted at:

https://sites.google.com/view/mibdataanalyticsassignment/home

This completes the end-to-end data product: from a user typing a city name into a Google Form, through the automated enrichment pipeline, to a polished interactive map card visible to anyone with the link. The entire stack remains within the Google ecosystem, requiring no external hosting, no custom servers, and no ongoing maintenance beyond the Apps Script trigger already in place.

Complete Pipeline Summary

The complete end-to-end pipeline, from user submission to fully enriched dashboard row, is visualised below. Each stage produces an output that feeds directly into the next, forming a seamless data product.

Conclusions

The Cities of the World Dashboard demonstrates that significant analytical capability can be delivered through judicious use of existing tooling, structured data design, and thoughtful automation, without requiring dedicated infrastructure, specialised engineering, or ongoing operational cost.

The key outcomes of the project are:

- **01** Operational leverage achieved by separating stable data (stored) from dynamic data (live), dramatically reducing API dependency and improving response time.
- **02** Full automation of a multi-step enrichment pipeline through Apps Script event triggers, eliminating all manual data entry and processing steps.
- **03** Transparent failure handling that surfaces errors explicitly rather than silently, improving dashboard auditability and team trust in the data.
- **04** Scalable architecture that can be extended with additional data sources, API integrations, or AI enrichments without restructuring the core pipeline.
- **05** Low operational cost by leveraging Google's native ecosystem, avoiding external servers, ETL infrastructure, or database management systems.

This project stands as a practical proof-of-concept for the principle that data engineering sophistication is not measured by infrastructure complexity (using multiple pieces) but by the clarity of the architecture, the reliability of the automation, and the quality of the resulting data product.